```java
public class Tester {
    public static void main(String[] args) {
        Graph t = new Graph();
    }
}
```

class tester เอาไว้ใช้ทดสอบ

```
=== Prim: Minimum Spanning Tree ===
  e1 -- e3  (weight: 2)
  e3 -- e2  (weight: 1)
  e3 -- e4  (weight: 3)

Total Weight: 6

-----------------------

=== Kruskal: Minimum Spanning Tree ===
  e2 -- e3  (weight: 1)
  e1 -- e3  (weight: 2)
  e3 -- e4  (weight: 3)

Total Weight: 6
```

```java
public class Graph {
    private String input;
    private Prim prim;
    private Kruskal krus;

    public Graph() {
        input = "e1-e2-4,e1-e3-2,e2-e3-1,e2-e4-5,e3-e4-3";
        prim = new Prim(input);
        krus = new Kruskal(input);
        showResult();
    }
    public Prim getPrim() {
        return prim;
    }
    public Kruskal getKruskal() {
        return krus;
    }
    public void setPrim(Prim p) {
        prim = p;
    }
    public void setKruskal(Kruskal k) {
        krus = k;
    }
    public void showResult() {
        prim.run();
        System.out.println("\n-----------------------\n");
        krus.run();
    }
}
```

class Graph เอาไว้เก็บ input และ class ที่เกี่ยวข้อง

attribute มี input ที่เก็บเป็น String
prim ที่ใช้เก็บ prim algorithm
krus ใช้เก็บ kruskal algorithm

Constructor รับค่า input และสร้าง object ที่เกี่ยวข้องพร้อมแสดงผลด้วย method showResult

getter ของ prim

getter ของ kruskal

setter ของ prim

getter ของ kruskal

method เอาไว้แสดงผลลัพธ์ทั้งหมด

```java
import java.util.*;

public class Prim {

    private String input;  // เช่น "e1-e2-4,e1-e3-2,e2-e3-1,e3-e4-3"

    private Map<String, List<int[]>> graph = new HashMap<>();
    private List<String> nodeList = new ArrayList<>();
    private Map<String, Integer> nodeIndex = new LinkedHashMap<>();

    public Prim(String input) {
        this.input = input;
    }

    public void parseInput() {
        String[] edges = input.split(",");
        for (String edge : edges) {
            String[] parts = edge.trim().split("-");
            if (parts.length != 3) {
                System.out.println("รูปแบบผิด: " + edge + " (ต้องเป็น node1-node2-weight)");
                continue;
            }
            String u = parts[0].trim();
            String v = parts[1].trim();
            int w;
            try {
                w = Integer.parseInt(parts[2].trim());
            } catch (NumberFormatException e) {
                System.out.println("weight ต้องเป็นตัวเลข: " + edge);
                continue;
            }

            if (!nodeIndex.containsKey(u)) { nodeIndex.put(u, nodeList.size()); nodeList.add(u); }
            if (!nodeIndex.containsKey(v)) { nodeIndex.put(v, nodeList.size()); nodeList.add(v); }

            // เดินไปกลับได้ (undirected)
            graph.computeIfAbsent(u, k -> new ArrayList<>()).add(new int[]{nodeIndex.get(v), w});
            graph.computeIfAbsent(v, k -> new ArrayList<>()).add(new int[]{nodeIndex.get(u), w});
        }
    }

    public void run() {
        parseInput();
        if (nodeList.isEmpty()) { System.out.println("ไม่มีข้อมูล"); return; }

        int n = nodeList.size();
        int[] key     = new int[n];
        int[] parent  = new int[n];
        boolean[] inMST = new boolean[n];

        Arrays.fill(key, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);
        key[0] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
        pq.offer(new int[]{0, 0});

        int totalWeight = 0;
        List<String> mstEdges = new ArrayList<>();

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int u = curr[1];
            if (inMST[u]) continue;
            inMST[u] = true;

            if (parent[u] != -1) {
                mstEdges.add(nodeList.get(parent[u]) + " -- " + nodeList.get(u) + "  (weight: " + key[u] + ")");
                totalWeight += key[u];
            }

            for (int[] neighbor : graph.getOrDefault(nodeList.get(u), new ArrayList<>())) {
                int v = neighbor[0], w = neighbor[1];
                if (!inMST[v] && w < key[v]) {
                    key[v] = w;
                    parent[v] = u;
                    pq.offer(new int[]{w, v});
                }
            }
        }

        System.out.println("=== Prim: Minimum Spanning Tree ===");
        mstEdges.forEach(e -> System.out.println("  " + e));
        System.out.println("\nTotal Weight: " + totalWeight);

        long visited = 0;
        for (boolean b : inMST) if (b) visited++;
        if (visited < n) System.out.println("⚠ กราฟไม่ connected (" + (n - visited) + " node ที่เข้าไม่ถึง)");
    }
    public String getInput() { return input; }
    public void setInput(String input) { this.input = input; }
}
```

class Prim เอาไว้ทำ prim algorithm

มี attribute input เป็น String
มี graph ใช้เก็บ Adjacency List ของกราฟ
มี nodeList ใช้เก็บชื่อ node ทั้งหมด
มี nodeIndex ใช้เก็บน้ำหนักของ node

Constructor ให้รับ input

method ใช้แปลง input ให้เป็น graph
edges เก็บ edge ทั้งหมดโดยการแยกด้วยเครื่องหมาย ","
วนลูปให้ครบทุก edge
parts เก็บ node กับ weight โดยตรวจด้วยว่ารูปแบบถูกต้องมั้ย

ให้ u เป็น node ต้นทางและ v เป็น node ปลายทาง
ให้ w เป็น weight

แปลงให้ weight เป็นตัวเลข

เพิ่ม node ใหม่หากยังไม่มี node นี้

สร้าง edge ใน graph โดยทำให้เดินไป-กลับได้

method ใช้เรียกทำงาน
เตรียมข้อมูล input ให้เป็น graph
ให้ n เก็บจำนวน node ทั้งหมด

key เก็บ weight ที่ดีที่สุด
parent เก็บ node ก่อนหน้า
inMST ใช้เช็คว่า node อยู่ใน MST หรือยัง

ตั้งค่าเริ่มต้น key กับ parent

สร้าง pq ใช้หา weight ที่น้อยที่สุด

สร้าง totalWeight เก็บน้ำหนักรวม
สร้าง mstEdges เก็บ edge ที่อยู่ใน MST

ลูปจนกว่า pq จะว่าง
ดึง weight ที่น้อยที่สุดมาก่อน แล้วหา node
ถ้า node อยู่ใน MST แล้วให้ข้าม
ถ้าไม่มีให้เพิ่มลง MST

ถ้ามี parent ให้แสดง edge และเพิ่มน้ำหนัก

ตรวจเพื่อนบ้านให้ v เป็นปลายทาง และ w เป็น weight
โดยเช็คว่าไม่อยู่ใน MST ใช่ไหมและ weight น้อยกว่าหรือไม่
หากใช่ให้อัปเดทค่าใหม่

แสดง edge ทั้งหมดและแสดงน้ำหนักรวม

ตรวจว่ากราฟต่อเนื่องไหม โดยหาว่า visited น้อยกว่า n ไหม

```java
import java.util.*;

public class Kruskal {

    private String input;
    private List<int[]> edges = new ArrayList<>();
    private List<String>         nodeList  = new ArrayList<>();
    private Map<String, Integer> nodeIndex = new LinkedHashMap<>();
    private int[] parent;
    private int[] rank;

    public Kruskal(String input) {
        this.input = input;
    }

    public void parseInput() {
        String[] rawEdges = input.split(",");
        for (String edge : rawEdges) {
            String[] parts = edge.trim().split("-");
            if (parts.length != 3) {
                System.out.println("Invalid format: " + edge + " (must be node1-node2-weight)");
                continue;
            }
            String u = parts[0].trim();
            String v = parts[1].trim();
            int w;
            try {
                w = Integer.parseInt(parts[2].trim());
            } catch (NumberFormatException e) {
                System.out.println("Weight must be a number: " + edge);
                continue;
            }

            if (!nodeIndex.containsKey(u)) { nodeIndex.put(u, nodeList.size()); nodeList.add(u); }
            if (!nodeIndex.containsKey(v)) { nodeIndex.put(v, nodeList.size()); nodeList.add(v); }

            edges.add(new int[]{ nodeIndex.get(u), nodeIndex.get(v), w });
        }
    }

    private int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    private boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;

        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }

    public void run() {
        parseInput();
        if (nodeList.isEmpty()) { System.out.println("No data found."); return; }

        int n = nodeList.size();

        parent = new int[n];
        rank   = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; rank[i] = 0; }

        edges.sort(Comparator.comparingInt(e -> e[2]));

        int totalWeight  = 0;
        int mstEdgeCount = 0;
        List<String> mstEdges = new ArrayList<>();

        for (int[] e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (union(u, v)) {
                mstEdges.add(nodeList.get(u) + " -- " + nodeList.get(v) + "  (weight: " + w + ")");
                totalWeight  += w;
                mstEdgeCount++;
            }
            if (mstEdgeCount == n - 1) break;
        }

        System.out.println("=== Kruskal: Minimum Spanning Tree ===");
        mstEdges.forEach(e -> System.out.println("  " + e));
        System.out.println("\nTotal Weight: " + totalWeight);


        Set<Integer> roots = new HashSet<>();
        for (int i = 0; i < n; i++) roots.add(find(i));
        if (roots.size() > 1)
            System.out.println("⚠ Graph is not connected (" + (roots.size() - 1) + " disconnected component(s))");
    }

    public String getInput()             { return input;       }
    public void   setInput(String input) { this.input = input; }
}
```

class Kruskal เอาไว้ทำ kruskal algorithm

attribute มี input ที่เก็บเป็น String
มี edges ใช้เก็บ edge ทั้งหมด
มี nodeList ใช้เก็บชื่อ node ทั้งหมด
มี nodeIndex ใช้เก็บน้ำหนักของ node
มี parent ใช้เก็บ node ที่ซ้ำ
มี rank ใช้เก็บความสูงของ tree

method ใช้แปลง input ให้เป็น graph
edges เก็บ edge ทั้งหมดโดยการแยกด้วยเครื่องหมาย ","
วนลูปให้ครบทุก edge
parts เก็บ node กับ weight โดยตรวจด้วยว่ารูปแบบถูกต้องมั้ย

ให้ u เป็น node ต้นทางและ v เป็น node ปลายทาง
ให้ w เป็น weight

แปลงให้ weight เป็นตัวเลข

เพิ่ม node ใหม่หากยังไม่มี node นี้

เพิ่ม edge ลงใน edges

method ใช้หา root ของ parent ทำให้ tree สั้นลง

method รวม set ของ x และ y
โดยหา root ของ x และ y
หากเป็น root ให้ return false เพราะจะเกิดเป็น cycle

หา tree อันที่สูงกว่าเพื่อให้เป็น parent แล้วนำมาเชื่อมกัน
แต่ถ้า rank เท่ากันให้เพิ่ม rank เพราะสูงเท่ากันมาเชื่อมกัน

method ใช้เรียกทำงาน
เตรียมข้อมูล input ให้เป็น graph

ให้ n เก็บจำนวน node ทั้งหมด

สร้าง parent และ rank

ลูปให้ทุก node เป็น root ของตัวเอง

เรียง edge ตาม weight
สร้างตัวแปรไว้เก็บ weight ทั้งหมด จำนวน edge ใน MST
และรายการ edge ใน MST

ลูป edge ทั้งหมด

เชื่อม node ถ้าไม่เกิด cycle
เพิ่ม edge เข้า MST เพิ่ม weight

หยุดเมื่อ MST ครบ

แสดง edge ทั้งหมดและแสดงน้ำหนักรวม

ตรวจว่ากราฟต่อเนื่องไหม โดยหาว่ามี root มากกว่า 1 ไหม